

Importing Libraries

```
In [6]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.style.use('dark_background')
import plotly.express as px
import plotly.io as pio
pio.templates.default = "plotly_dark"
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

Reading CSV

```
In [7]: df = pd.read_csv('/content/zomato.csv', engine='python', on_bad_lines='skip')
df.head()
```

```
Out[7]:
```

	url	address	name	online_order	book_table	ra
0	https://www.zomato.com/bangalore/jalsa-banasha...	942, 21st Main Road, 2nd Stage, Banashankari, ...	Jalsa	Yes	Yes	4.
1	https://www.zomato.com/bangalore/spice-elephan...	2nd Floor, 80 Feet Road, Near Big Bazaar, 6th ...	Spice Elephant	Yes	No	4.
2	https://www.zomato.com/SanchurroBangalore?cont...	1112, Next to KIMS Medical College, 17th Cross...	San Churro Cafe	Yes	No	3.
3	https://www.zomato.com/bangalore/addhuri-udupi...	1st Floor, Annakuteera, 3rd Stage, Banashankar...	Addhuri Udupi Bhojana	No	No	3.
4	https://www.zomato.com/bangalore/grand-village...	10, 3rd Floor, Lakshmi Associates, Gandhi Baza...	Grand Village	No	No	3.

```
In [8]: df.shape
```

Out[8]: (3978, 17)

```
In [9]: df.columns
```

```
Out[9]: Index(['url', 'address', 'name', 'online_order', 'book_table', 'rate', 'votes',  
              'phone', 'location', 'rest_type', 'dish_liked', 'cuisines',  
              'approx_cost(for two people)', 'reviews_list', 'menu_item',  
              'listed_in(type)', 'listed_in(city)'],  
              dtype='object')
```

```
In [10]: df = df.drop(['url', 'address', 'phone', 'menu_item', 'dish_liked', 'reviews_l  
df.head()
```

```
Out[10]:
```

	name	online_order	book_table	rate	votes	location	rest_type	cui
0	Jalsa	Yes	Yes	4.1/5	775	Banashankari	Casual Dining	I Mu Ch
	Elephant							Chi
2	San Churro Cafe	Yes	No	3.8/5	918	Banashankari	Cafe, Casual Dining	Me I
3	Addhuri Udupi Bhojana	No	No	3.7/5	88	Banashankari	Quick Bites	I I
4	Grand Village	No	No	3.8/5	166	Basavanagudi	Casual Dining	I Raja

```
In [11]: df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3978 entries, 0 to 3977
Data columns (total 11 columns):
#   Column                                          Non-Null Count  Dtype
---  -
0   name                                           3978 non-null   object
1   online_order                                  3978 non-null   object
2   book_table                                    3978 non-null   object
3   rate                                           3468 non-null   object
4   votes                                          3978 non-null   int64
5   location                                      3977 non-null   object
6   rest_type                                    3964 non-null   object
7   cuisines                                      3974 non-null   object
8   approx_cost(for two people)                 3972 non-null   object
9   listed_in(type)                             3978 non-null   object
10  listed_in(city)                             3978 non-null   object
dtypes: int64(1), object(10)
memory usage: 342.0+ KB

```

Dropping Duplicates

```

In [12]: df.drop_duplicates(inplace = True)
df.shape

```

```

Out[12]: (3975, 11)

```

Cleaning Rate Column

```

In [13]: df['rate'].unique()

```

```

Out[13]: array(['4.1/5', '3.8/5', '3.7/5', '3.6/5', '4.6/5', '4.0/5', '4.2/5',
                '3.9/5', '3.1/5', '3.0/5', '3.2/5', '3.3/5', '2.8/5', '4.4/5',
                '4.3/5', 'NEW', '2.9/5', '3.5/5', nan, '2.6/5', '3.8 /5', '3.4/5',
                '4.5/5', '2.5/5', '2.7/5', '4.7/5', '2.4/5', '2.2/5', '2.3/5',
                '3.4 /5', '-', '3.6 /5', '4.8/5', '3.9 /5', '4.2 /5', '4.0 /5',
                '4.1 /5', '3.7 /5', '3.1 /5', '2.9 /5', '3.3 /5', '2.8 /5',
                '3.5 /5', '2.7 /5', '2.5 /5', '3.2 /5', '2.6 /5', '4.5 /5',
                '4.3 /5', '4.4 /5', '4.9/5'], dtype=object)

```

Removing "NEW" , "-" and "/5" from Rate Column

```

In [14]: def handlerate(value):
            if(value=='NEW' or value=='-'):
                return np.nan
            else:
                value = str(value).split('/')
                value = value[0]
                return float(value)

```

```
df['rate'] = df['rate'].apply(handlerate)
df['rate'].head()
```

```
Out[14]:
```

	rate
0	4.1
1	4.1
2	3.8
3	3.7
4	3.8

dtype: float64

Filling Null Values in Rate Column with Mean

```
In [15]: df['rate'].fillna(df['rate'].mean(), inplace = True)
df['rate'].isnull().sum()
```

```
Out[15]: np.int64(0)
```

```
In [16]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 3975 entries, 0 to 3977
Data columns (total 11 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   name                                  3975 non-null   object
1   online_order                          3975 non-null   object
2   book_table                            3975 non-null   object
3   rate                                  3975 non-null   float64
4   votes                                 3975 non-null   int64
5   location                              3974 non-null   object
6   rest_type                             3961 non-null   object
7   cuisines                              3971 non-null   object
8   approx_cost(for two people)           3969 non-null   object
9   listed_in(type)                       3975 non-null   object
10  listed_in(city)                       3975 non-null   object
dtypes: float64(1), int64(1), object(9)
memory usage: 372.7+ KB
```

Dropping Null Values

```
In [17]: df.dropna(inplace = True)
df.head()
```

Out[17]:

	name	online_order	book_table	rate	votes	location	rest_type	cuis
0	Jalsa	Yes	Yes	4.1	775	Banashankari	Casual Dining	In Mu Ch
1	Spice Elephant	Yes	No	4.1	787	Banashankari	Casual Dining	Chi In
2	San Churro Cafe	Yes	No	3.8	918	Banashankari	Cafe, Casual Dining	Mex It
3	Addhuri Udupi Bhojana	No	No	3.7	88	Banashankari	Quick Bites	S In I
4	Grand Village	No	No	3.8	166	Basavanagudi	Casual Dining	In Rajas

```
In [18]: df.rename(columns = {'approx_cost(for two people)': 'Cost2plates', 'listed_in(t
df.head()
```

Out[18]:

	name	online_order	book_table	rate	votes	location	rest_type	cuis
0	Jalsa	Yes	Yes	4.1	775	Banashankari	Casual Dining	In Mu Ch
1	Spice Elephant	Yes	No	4.1	787	Banashankari	Casual Dining	Chi In
2	San Churro Cafe	Yes	No	3.8	918	Banashankari	Cafe, Casual Dining	Mex It
3	Addhuri Udupi Bhojana	No	No	3.7	88	Banashankari	Quick Bites	S In I
4	Grand Village	No	No	3.8	166	Basavanagudi	Casual Dining	In Rajas

```
In [19]: df['location'].unique()
```

```
Out[19]: array(['Banashankari', 'Basavanagudi', 'Mysore Road', 'Jayanagar',
               'Kumaraswamy Layout', 'Rajarajeshwari Nagar', 'Vijay Nagar',
               'Uttarahalli', 'JP Nagar', 'South Bangalore', 'City Market',
               'Nagarbhavi', 'Bannerghatta Road', 'BTM', 'Kanakapura Road',
               'Bommanahalli', 'CV Raman Nagar', 'Electronic City', 'HSR',
               'Marathahalli', 'Wilson Garden', 'Shanti Nagar',
               'Koramangala 5th Block', 'Koramangala 8th Block', 'Richmond Road',
               'Koramangala 7th Block', 'Jalahalli', 'Koramangala 4th Block',
               'Bellandur', 'Sarjapur Road', 'Whitefield', 'East Bangalore',
               'Old Airport Road'], dtype=object)
```

```
In [20]: df['listed_in(city)'].unique()
```

```
Out[20]: array(['Banashankari', 'Bannerghatta Road', 'Basavanagudi', 'Bellandur'],
               dtype=object)
```

Listed in(city) and location, both are there, lets keep only one.

```
In [21]: df = df.drop(['listed_in(city)'], axis = 1)
```

```
In [22]: df['Cost2plates'].unique()
```

```
Out[22]: array(['800', '300', '600', '700', '550', '500', '450', '650', '400',
               '900', '200', '750', '150', '850', '100', '1,200', '350', '250',
               '950', '1,000', '1,500', '1,300', '199', '80', '1,100', '160',
               '1,600', '230', '130', '50', '190', '1,700', '1,400', '180',
               '1,350', '2,200', '2,000', '1,800', '1,900'], dtype=object)
```

Removing , from Cost2Plates Column

```
In [23]: def handlecomma(value):
          value = str(value)
          if ',' in value:
              value = value.replace(',', '')
              return float(value)
          else:
              return float(value)

          df['Cost2plates'] = df['Cost2plates'].apply(handlecomma)
          df['Cost2plates'].unique()
```

```
Out[23]: array([ 800.,  300.,  600.,  700.,  550.,  500.,  450.,  650.,  400.,
                900.,  200.,  750.,  150.,  850.,  100., 1200.,  350.,  250.,
                950., 1000., 1500., 1300.,  199.,   80., 1100.,  160., 1600.,
                230.,  130.,   50.,  190., 1700., 1400.,  180., 1350., 2200.,
                2000., 1800., 1900.]
```

```
In [24]: df.head()
```

Out [24]:

	name	online_order	book_table	rate	votes	location	rest_type	cuis
0	Jalsa	Yes	Yes	4.1	775	Banashankari	Casual Dining	In Mu Ch
1	Spice Elephant	Yes	No	4.1	787	Banashankari	Casual Dining	Chi In
2	San Churro Cafe	Yes	No	3.8	918	Banashankari	Cafe, Casual Dining	Mex It
3	Addhuri Udupi Bhojana	No	No	3.7	88	Banashankari	Quick Bites	S In I
4	Grand Village	No	No	3.8	166	Basavanagudi	Casual Dining	In Rajas

Cleaning Rest Type Column

```
In [25]: rest_types = df['rest_type'].value_counts(ascending = False)
rest_types
```

Out[25]:

rest_type	count
Quick Bites	1680
Casual Dining	824
Cafe	293
Dessert Parlor	208
Delivery	160
Takeaway, Delivery	119
Bakery	109
Beverage Shop	68
Food Court	54
Sweet Shop	48
Casual Dining, Bar	42
Beverage Shop, Quick Bites	27
Bar	26
Bakery, Quick Bites	22
Casual Dining, Cafe	21
Cafe, Casual Dining	19
Sweet Shop, Quick Bites	17
Pub	16
Lounge	15
Food Truck	14
Quick Bites, Cafe	14
Takeaway	13
Kiosk	12
Mess	12
Bar, Casual Dining	9
Microbrewery, Casual Dining	9
Beverage Shop, Dessert Parlor	9
Quick Bites, Dessert Parlor	8
Cafe, Quick Bites	8
Pub, Casual Dining	7

	count
rest_type	
Quick Bites, Beverage Shop	6
Cafe, Bakery	6
Confectionery	5
Bakery, Dessert Parlor	5
Casual Dining, Pub	5
Microbrewery	4
Dessert Parlor, Sweet Shop	4
Dessert Parlor, Beverage Shop	4
Pub, Bar	3
Quick Bites, Bakery	3
Dessert Parlor, Quick Bites	3
Fine Dining	2
Dhaba	2
Lounge, Bar	2
Food Court, Quick Bites	2
Beverage Shop, Cafe	2
Casual Dining, Sweet Shop	2
Quick Bites, Sweet Shop	1
Food Court, Dessert Parlor	1
Food Court, Casual Dining	1
Casual Dining, Microbrewery	1
Bakery, Beverage Shop	1
Sweet Shop, Dessert Parlor	1
Lounge, Casual Dining	1
Cafe, Food Court	1
Cafe, Dessert Parlor	1
Dessert Parlor, Cafe	1

dtype: int64

```
In [26]: rest_types_lessthan1000 = rest_types[rest_types<1000]
```

```
rest_types_lessthan1000
```

Out[26]:

	count
rest_type	
Casual Dining	824
Cafe	293
Dessert Parlor	208
Delivery	160
Takeaway, Delivery	119
Bakery	109
Beverage Shop	68
Food Court	54
Sweet Shop	48
Casual Dining, Bar	42
Beverage Shop, Quick Bites	27
Bar	26
Bakery, Quick Bites	22
Casual Dining, Cafe	21
Cafe, Casual Dining	19
Sweet Shop, Quick Bites	17
Pub	16
Lounge	15
Food Truck	14
Quick Bites, Cafe	14
Takeaway	13
Kiosk	12
Mess	12
Bar, Casual Dining	9
Microbrewery, Casual Dining	9
Beverage Shop, Dessert Parlor	9
Quick Bites, Dessert Parlor	8
Cafe, Quick Bites	8
Pub, Casual Dining	7
Quick Bites, Beverage Shop	6

	count
rest_type	
Cafe, Bakery	6
Confectionery	5
Bakery, Dessert Parlor	5
Casual Dining, Pub	5
Microbrewery	4
Dessert Parlor, Sweet Shop	4
Dessert Parlor, Beverage Shop	4
Pub, Bar	3
Quick Bites, Bakery	3
Dessert Parlor, Quick Bites	3
Fine Dining	2
Dhaba	2
Lounge, Bar	2
Food Court, Quick Bites	2
Beverage Shop, Cafe	2
Casual Dining, Sweet Shop	2
Quick Bites, Sweet Shop	1
Food Court, Dessert Parlor	1
Food Court, Casual Dining	1
Casual Dining, Microbrewery	1
Bakery, Beverage Shop	1
Sweet Shop, Dessert Parlor	1
Lounge, Casual Dining	1
Cafe, Food Court	1
Cafe, Dessert Parlor	1
Dessert Parlor, Cafe	1

dtype: int64

Making Rest Types less than 1000 in frequency as others

```
In [27]: def handle_rest_type(value):
          if (value in rest_types_lessthan1000):
              return 'others'
          else:
              return value

df['rest_type'] = df['rest_type'].apply(handle_rest_type)
df['rest_type'].value_counts()
```

Out[27]:

	count
rest_type	
others	2273
Quick Bites	1680

rest_type	
others	2273
Quick Bites	1680

dtype: int64

Cleaning Location Column

```
In [28]: location = df['location'].value_counts(ascending = False)

location_lessthan300 = location[location < 300]

df['location'] = df['location'].apply(
    lambda x: 'others' if x in location_lessthan300 else x
)
```

Cleaning Cuisines Column

```
In [29]: cuisines = df['cuisines'].value_counts(ascending = False)

cuisines_lessthan100 = cuisines[cuisines < 100]

def handle_cuisines(value):
    if (value in cuisines_lessthan100):
        return 'others'
    else:
        return value

df['cuisines'] = df['cuisines'].apply(handle_cuisines)
```

```
df['cuisines'].value_counts()
```

```
Out[29]:
```

	count
cuisines	
others	3306
South Indian	244
North Indian	203
North Indian, Chinese	200

dtype: int64

```
In [30]: df.head()
```

```
Out[30]:
```

	name	online_order	book_table	rate	votes	location	rest_type	cuisi
0	Jalsa	Yes	Yes	4.1	775	Banashankari	others	ot
1	Spice Elephant	Yes	No	4.1	787	Banashankari	others	ot
2	San Churro Cafe	Yes	No	3.8	918	Banashankari	others	ot
3	Addhuri Udupi Bhojana	No	No	3.7	88	Banashankari	Quick Bites	ot
4	Grand Village	No	No	3.8	166	Basavanagudi	others	ot

Data Cleaning Done!, Lets jump to Visualization.

Count Plot of Various Locations

```
In [31]: fig = px.histogram(df, y='location', color='online_order', orientation='h',  
  
    labels={  
        'location': 'Restaurant Location',  
        'online_order': 'Online Order Available'  
    },  
  
    color_discrete_map={'Yes': '#00CC96', 'No': '#EF553B'},  
  
    title='Online Order Availability by Location'  
)
```

```
fig.update_layout(
    barmode='stack',
    font=dict(family='Montserrat'),

    xaxis_title='Restaurant Count',
    yaxis_title='Location'
)

fig.show()
```



Visualizing Online Order

```
In [32]: fig = px.histogram(df, y='online_order', orientation='h', color='online_order',
    color_discrete_sequence=['#19D3F3', '#636EFA'])

fig.update_layout(font=dict(family='Montserrat'),

    title='Online Order Count',
    legend_title='Online Order',
```

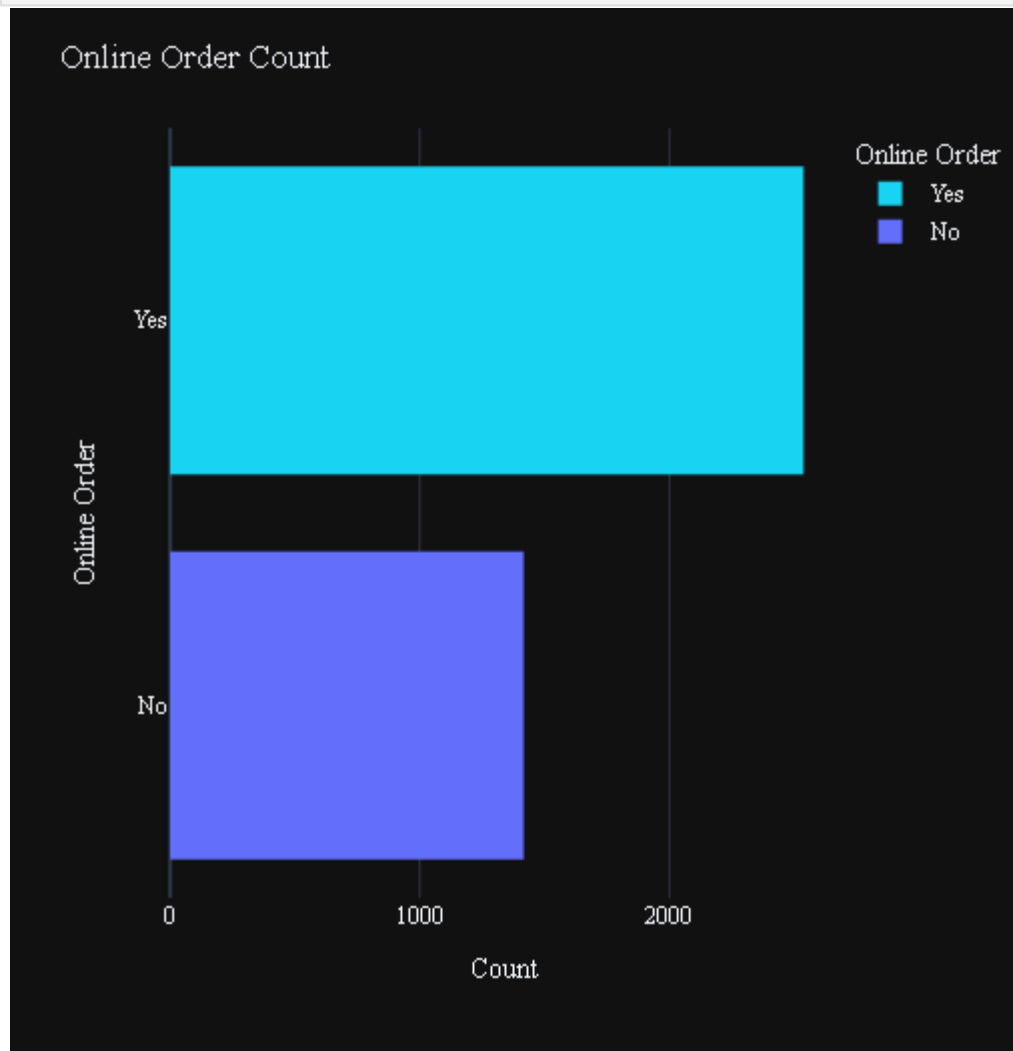
```

yaxis_title='Online Order',
xaxis_title='Count',

)

fig.show()

```



Visualizing Book Table

```

In [33]: fig = px.histogram( df,y='book_table', color='book_table',
                             color_discrete_map={ 'Yes': '#00E5FF', 'No': '#FFA15A' }
                             )

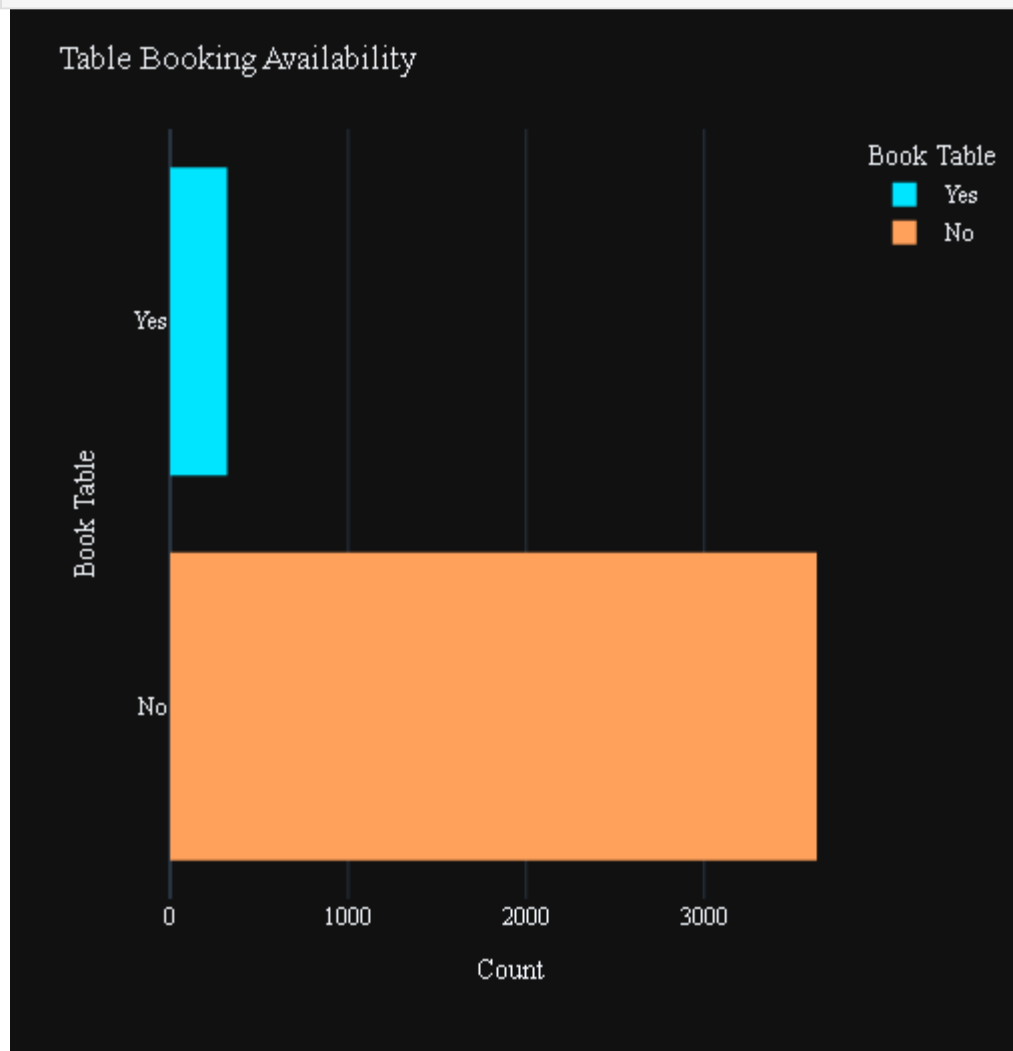
fig.update_layout( font=dict( family='Montserrat'),

                    title='Table Booking Availability',
                    legend_title='Book Table',

                    xaxis_title='Count',
                    yaxis_title='Book Table'

```

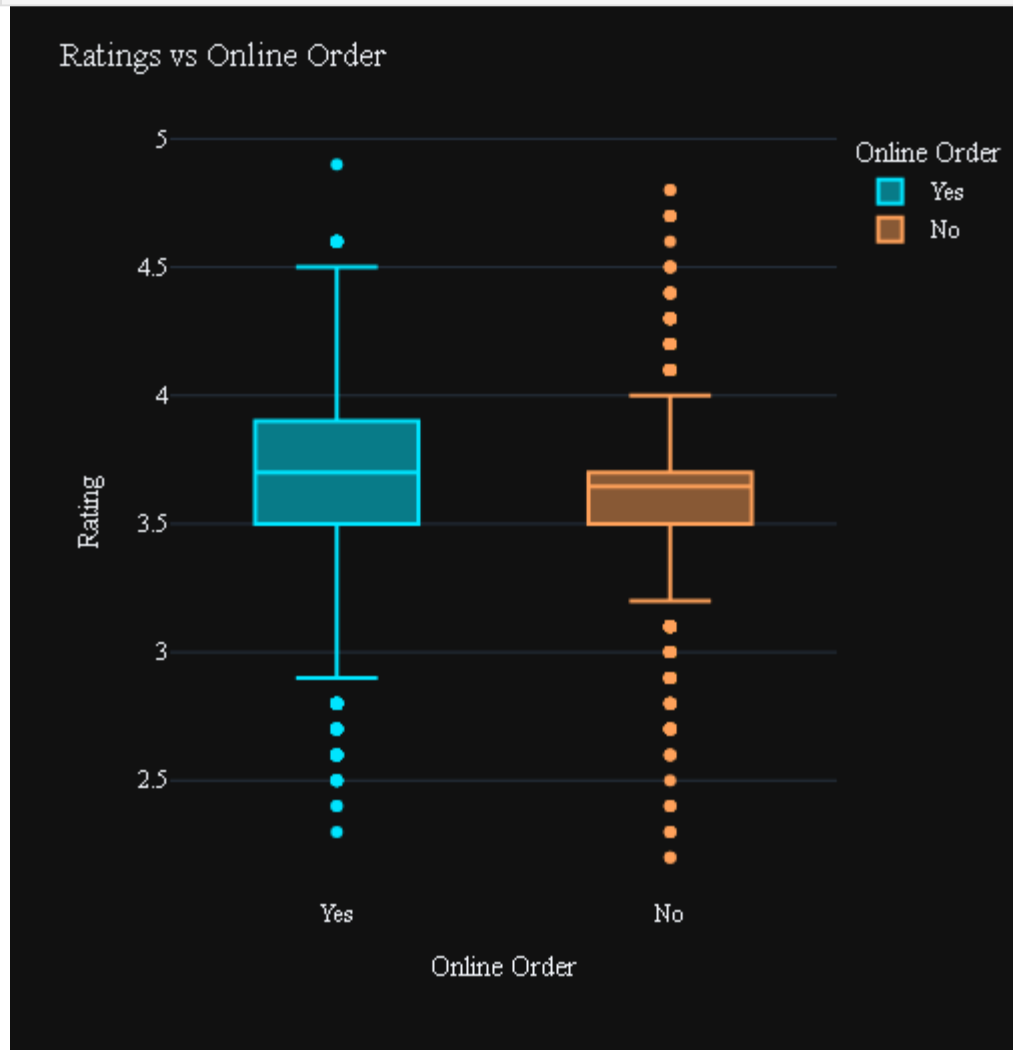
```
)  
  
fig.show()
```



Visualizing Online Order vs Rate

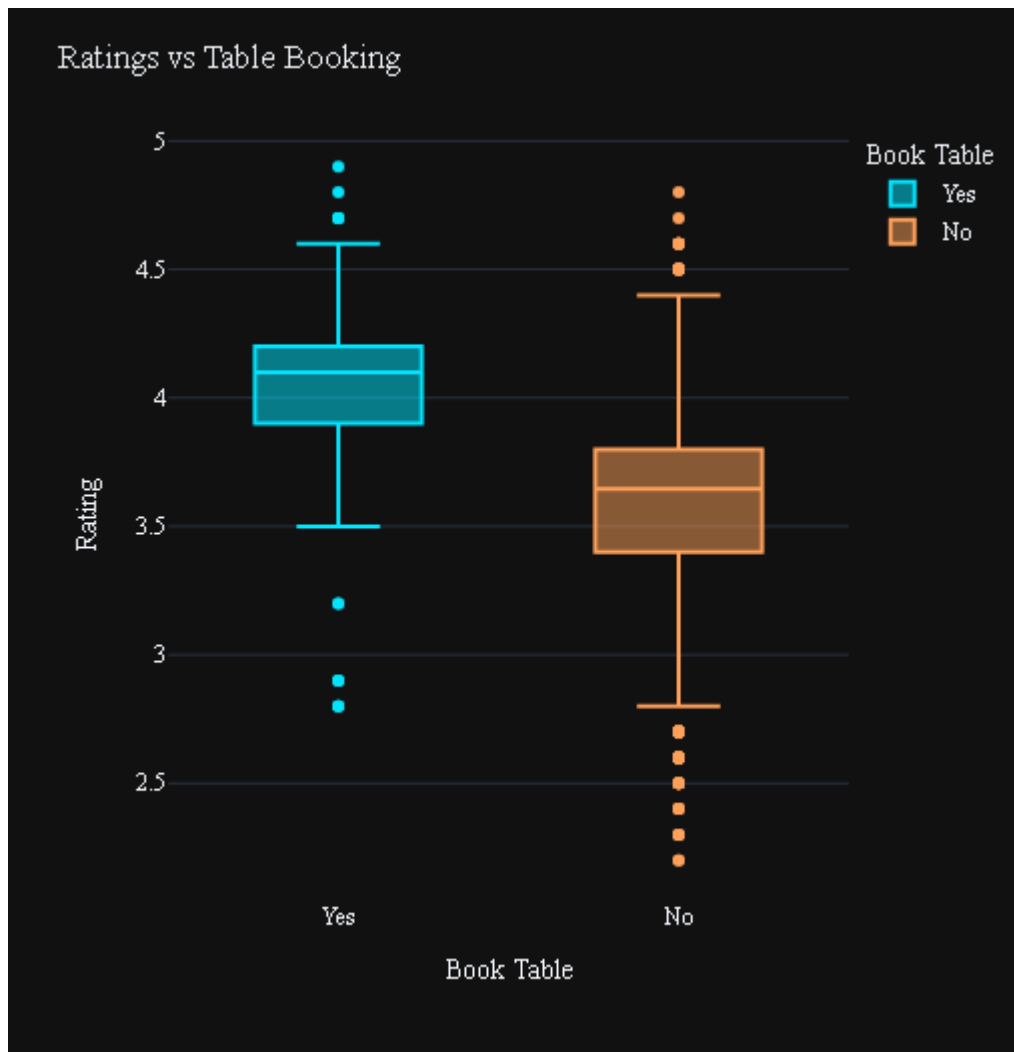
```
In [34]: fig = px.box(df, x='online_order', y='rate', color='online_order',  
  
    color_discrete_map={ 'Yes': '#00E5FF', 'No': '#FFA15A'})  
  
fig.update_layout(font=dict(family='Montserrat'),  
  
    title='Ratings vs Online Order',  
    legend_title='Online Order',  
  
    xaxis_title='Online Order',  
    yaxis_title='Rating'  
)
```

```
fig.show()
```



Visualizing Book Table vs Rate

```
In [35]: fig = px.box(df, x='book_table', y='rate', color='book_table',  
  
    color_discrete_map={'Yes': '#00E5FF', 'No': '#FFA15A'})  
  
fig.update_layout(font=dict(family='Montserrat'),  
  
    title='Ratings vs Table Booking',  
    legend_title='Book Table',  
  
    xaxis_title='Book Table',  
    yaxis_title='Rating'  
)  
  
fig.show()
```



Visualizing Online Order Facility, Location Wise

```
In [36]: df1 = df.groupby(['location', 'online_order'])['name'].count()
df1.to_csv('location_online.csv')
df1 = pd.read_csv('location_online.csv')
df1 = pd.pivot_table(df1, values=None, index=['location'], columns=['online_or
df1
```

Out[36]:

	name	
	online_order	No Yes
location		
Banashankari	359	400
Bannerghatta Road	334	425
Basavanagudi	196	315
JP Nagar	203	390
Jayanagar	158	391
others	166	616

In [37]:

```
plt.style.use('dark_background')

ax = df1.plot(
    kind='bar',
    figsize=(20,12),
    color=['#FFA15A', '#00E5FF']
)

plt.title('Online Order Availability by Location')

plt.xlabel('Location')

plt.ylabel('Restaurant Count')

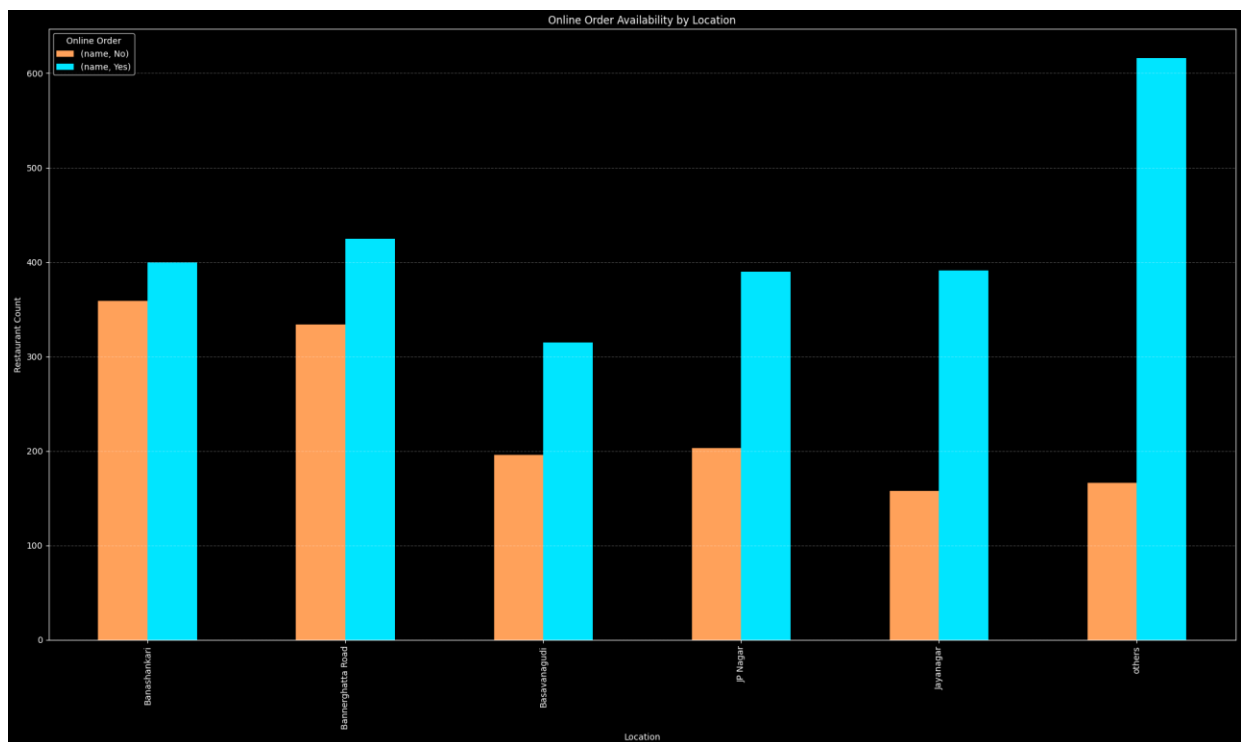
plt.xticks(rotation=90)

plt.legend(title='Online Order')

plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()

plt.show()
```



Visualizing Book Table Facility, Location Wise

```
In [38]: df2 = df.groupby(['location', 'book_table'])['name'].count()
df2.to_csv('location_booktable.csv')
df2 = pd.read_csv('location_booktable.csv')
df2 = pd.pivot_table(df2, values=None, index=['location'], columns=['book_table'])
df2
```

```
Out[38]:
```

location	name	
	book_table	No Yes
Banashankari	719	40
Bannerghatta Road	720	39
Basavanagudi	504	7
JP Nagar	512	81
Jayanagar	448	101
others	730	52

```
In [39]: plt.style.use('dark_background')

ax = df2.plot(
    kind='bar',
```

```

figsize=(15,8),
color=['#FFA15A', '#00E5FF']
)

plt.title('Restaurant Analysis by Location')
plt.xlabel('Location')
plt.ylabel('Count')

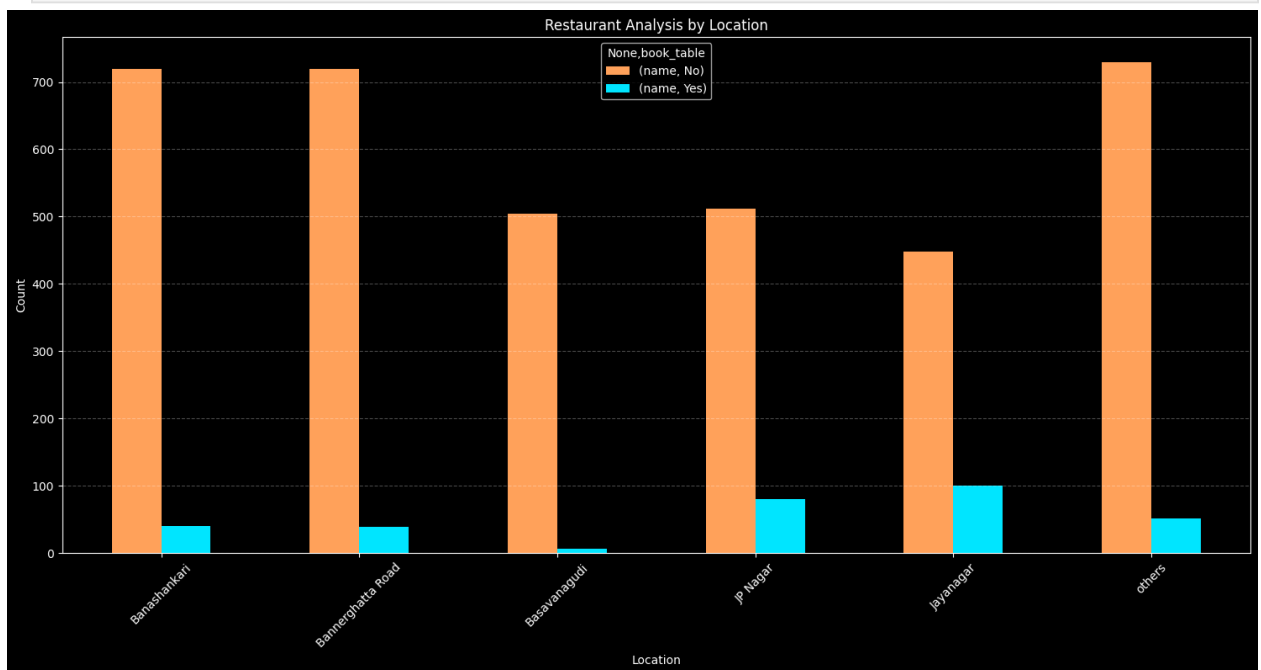
plt.xticks(rotation=45)

plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()

plt.show()

```



Visualizing Types of Restaurants vs Rate

```

In [40]: plt.figure(figsize=(14,8))

sns.boxplot(
    x='Type',
    y='rate',
    data=df,
    palette=['#FFA15A', '#00E5FF']
)

plt.title('Restaurant Type vs Rating')

plt.xlabel('Restaurant Type')

plt.ylabel('Rating')

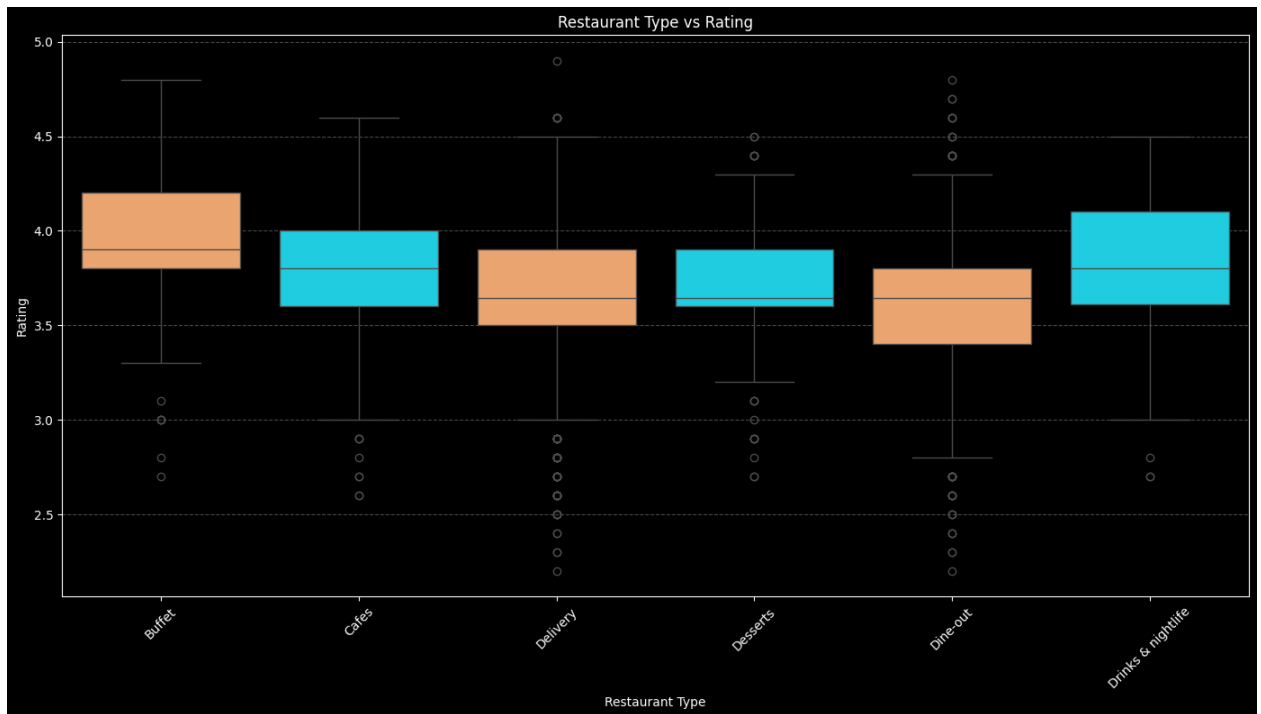
```

```
plt.xticks(rotation=45)

plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()

plt.show()
```



Grouping Types of Restaurants, location wise

```
In [41]: df3 = df.groupby(['location', 'Type'])['name'].count()
df3.to_csv('location_Type.csv')
df3 = pd.read_csv('location_Type.csv')
df3 = pd.pivot_table(df3, values=None, index=['location'], columns=['Type'], f
df3
```

Out[41]:

	Type	Buffet	Cafes	Delivery	Desserts	Dine-out	name Drinks & nightlife
location							
Banashankari		6	31	332	61	317	12
Bannerghatta Road		5	22	369	71	287	5
Basavanagudi		5	10	217	59	217	3
JP Nagar		11	19	363	37	151	12
Jayanagar		9	27	316	45	147	5
others		31	30	576	11	133	1

```
In [42]: plt.style.use('dark_background')

ax = df3.plot(
    kind='bar',
    figsize=(24,10),
    color=['#FFA15A', '#00E5FF']
)

plt.title('Restaurant Analysis')

plt.xlabel('Category')

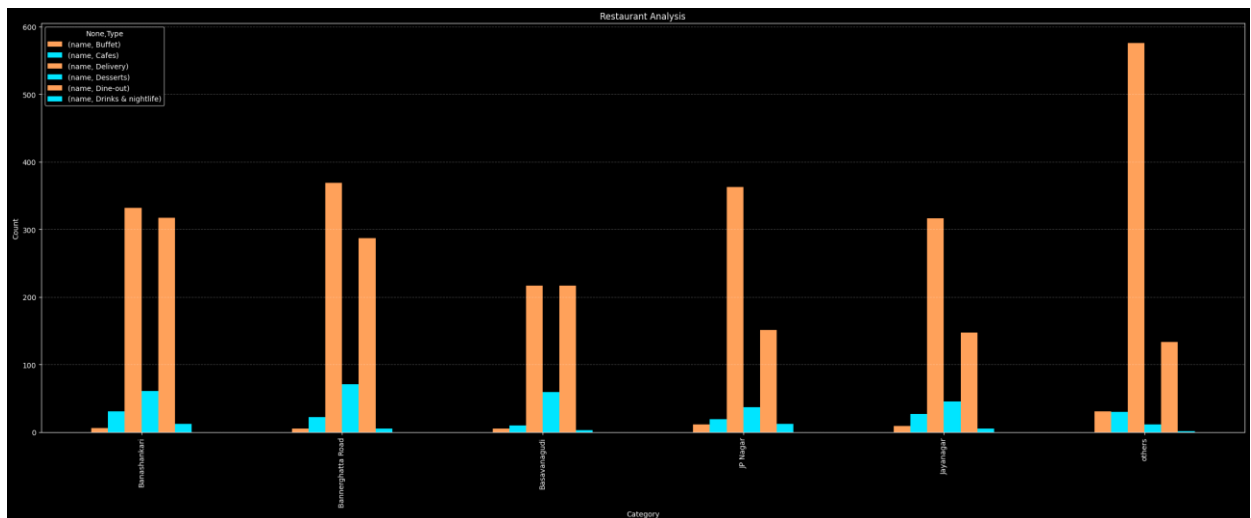
plt.ylabel('Count')

plt.xticks(rotation=90)

plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()

plt.show()
```



No. of Votes, Location Wise

```
In [43]: df4 = df[['location', 'votes']]
df4.drop_duplicates()
df5 = df4.groupby(['location'])['votes'].sum()
df5 = df5.to_frame()
df5 = df5.sort_values('votes', ascending=False)
df5.head()
```

```
Out[43]:
```

location	votes
Jayanagar	178973
others	161372
JP Nagar	147025
Banashankari	131818
Bannerghatta Road	83775

```
In [44]: plt.figure(figsize=(15,8))

sns.barplot(
    x=df5.index,
    y=df5['votes'],
    palette=['#FFA15A', '#00E5FF']
)

plt.title('Votes by Restaurant')

plt.xlabel('Restaurant')

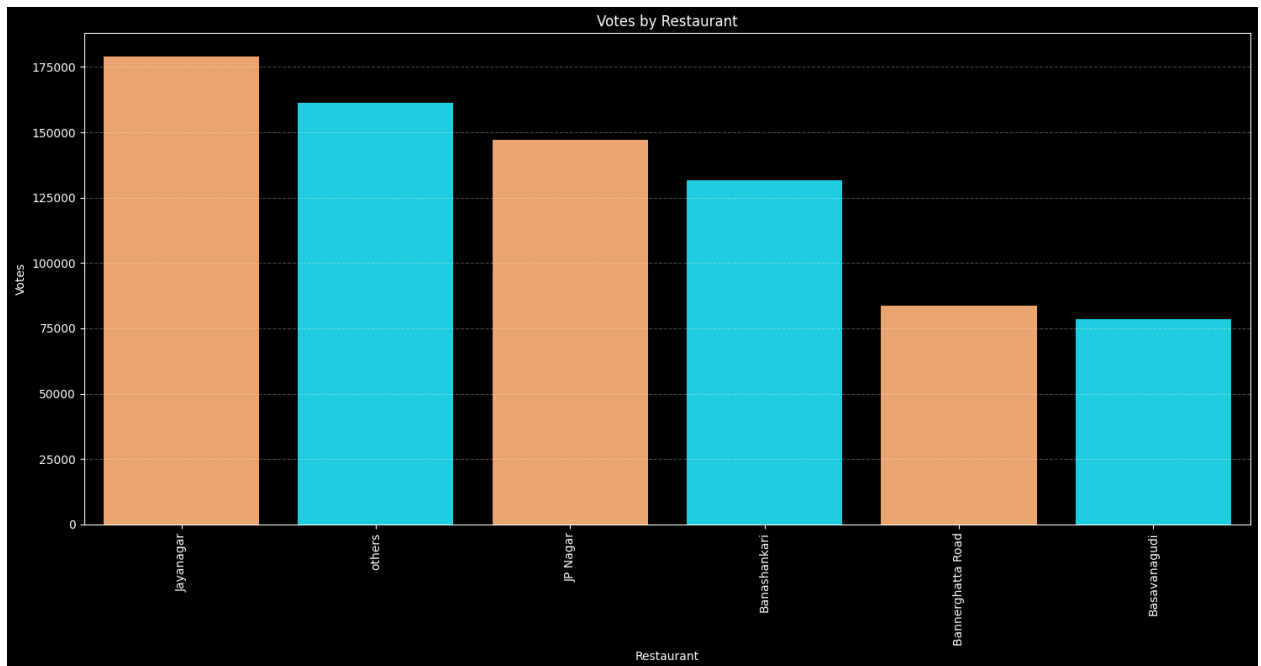
plt.ylabel('Votes')
```

```
plt.xticks(rotation=90)

plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()

plt.show()
```



In [45]: `df.head()`

Out [45]:

	name	online_order	book_table	rate	votes	location	rest_type	cuisi
0	Jalsa	Yes	Yes	4.1	775	Banashankari	others	ot
1	Spice Elephant	Yes	No	4.1	787	Banashankari	others	ot
2	San Churro Cafe	Yes	No	3.8	918	Banashankari	others	ot
3	Addhuri Udupi Bhojana	No	No	3.7	88	Banashankari	Quick Bites	ot
4	Grand Village	No	No	3.8	166	Basavanagudi	others	ot

Visualizing Top Cuisines

In [46]: `df6 = df[['cuisines', 'votes']]`
`df6.drop_duplicates()`

```
df7 = df6.groupby(['cuisines'])['votes'].sum()
df7 = df7.to_frame()
df7 = df7.sort_values('votes', ascending=False)
df7.head()
```

Out[46]:

votes	
cuisines	
others	679337
North Indian	39746
South Indian	31321
North Indian, Chinese	31190

In [47]:

```
df7 = df7.iloc[1:, :]
df7.head()
```

Out[47]:

votes	
cuisines	
North Indian	39746
South Indian	31321
North Indian, Chinese	31190

In [48]:

```
plt.figure(figsize=(15,8))

sns.barplot(
    x=df7.index,
    y=df7['votes'],
    palette=['#FFA15A', '#00E5FF']
)

plt.title('Votes Analysis')

plt.xlabel('Category')

plt.ylabel('Votes')

plt.xticks(rotation=90)

plt.grid(axis='y', linestyle='--', alpha=0.3)

plt.tight_layout()

plt.show()
```

